
Designing Razorpay's AI Revenue Recovery Agent

Executive Summary

- **Company Fit:** Razorpay Payments Private Limited is an RBI-authorised Payment Aggregator whose platform helps merchants, schools, ecommerce businesses, and other companies accept and disburse online payments, collect recurring payments, send invoices, and access business-finance products [7] -> design for merchant revenue recovery across Razorpay's payment surfaces, not only for Razorpay's own receivables.
- **Track Alignment:** The official brief asks builders to detect revenue at risk, select the right intervention, execute a bounded workflow, and show measured money recovered with compliant escalation, stopping rules, and an audit trail [8] -> build an outcome-measured recovery control plane rather than a conversational demo.
- **Direct Business Value:** A successful agent can turn failed payments, abandoned checkouts, subscription retries, and unpaid payment links into recoverable cases. The direct recovered amount belongs to the merchant, while the likely platform value is stronger merchant conversion, retention, and differentiation; Razorpay's internal fee and retention impact is not publicly established -> measure both merchant recovery and platform adoption hypotheses separately.
- **Best MVP:** Start with payment failure plus abandoned checkout, then add subscription recovery and overdue payment-link or invoice workflows. Razorpay documents failed-payment events, an abandoned-cart webhook, subscription retry behavior, payment-link creation, and reminder

management [43] [5] [45] [3] -> prove one reliable cross-surface loop before expanding.

- **Core Technical Insight:** Razorpay says webhook payloads are snapshots and that a payment can move from authorized to captured quickly, with event payloads reflecting the state at event time [43] -> store every event, deduplicate it, reconcile against current entity state, and never let an old webhook trigger a duplicate recovery action.
- **AI Boundary:** Stripe's Smart Retries show why timing and contextual signals can improve retry decisions, but they are a separate platform's capabilities and should be treated as design inspiration, not as Razorpay functionality [18] [18] -> let AI classify, prioritize, and draft within an allowlist while deterministic code controls money movement.
- **Safety Advantage:** Razorpay's Agent Studio principles describe review-first mode, merchant-defined permissions, scope checks, approval for irreversible actions, consent before outbound communication, dark-pattern screening, and full action logging [47] [47] -> make these visible product features in the project, not hidden implementation details.
- **Winning Demonstration:** The buildathon asks for a public repository, architecture, and a five-minute pitch, and evaluates problem taste, build quality, AI judgment, and failure recovery [8] -> show a replayable batch with ground truth, recovered value, non-actions, escalations, duplicate-event handling, and an audit trail.

Razorpay's Payments-to-Finance Platform and the Beneficiary

Key takeaway: Razorpay is a multi-surface business-finance platform, so recovery should be designed as a merchant operations capability that spans the transaction lifecycle.

Razorpay describes Razorpay Payments Private Limited as an RBI-authorised Payment Aggregator. Its stated proposition is developer-friendly payment acceptance and disbursement for merchants, schools, ecommerce businesses, and other companies, with credit cards, debit cards, netbanking, UPI, and wallets among the

supported modes [7] [7]. This matters because a recovery system sits inside a regulated payment flow, not beside an ordinary ecommerce CRM.

The platform also reaches beyond checkout. Razorpay says businesses can manage marketplaces, automate bank transfers, collect recurring payments, share invoices, and access working-capital loans from one platform [7]. RazorpayX adds business banking, corporate cards, payroll, vendor payments, and API solutions, with separate relevance for startups, SMEs, and enterprises [38] [38]. The company therefore has the ingredients for a unified recovery view: payment attempts, recurring obligations, receivables, and operational cash flows.

The important scope distinction is who owns the recovered money. In a merchant payment failure, the merchant generally has the immediate commercial loss; the project should not claim that every recovered rupee is Razorpay revenue. The strategic hypothesis is indirect: if Razorpay helps merchants convert more attempts into captures and collect more invoices, Razorpay can make its payment and finance platform more valuable. That hypothesis needs merchant cohort data, pricing, retention, and fee information that the public sources do not provide.

Razorpay surface	Revenue event	Proposed recovery value	Direct control boundary
Payment Gateway	Failed or incomplete payment attempt	Diagnose, retry where permitted, or reopen checkout	Never infer success from a failure notification; verify current payment and order state [19] [43]
Subscriptions	Failed automatic charge and pending subscription	Sequence retries, request customer action, or escalate	Use documented subscription retry and webhook behavior; do not invent terminal states [45] [4]
Payment Links and Invoices	Link sent but unpaid or approaching expiry	Send a controlled reminder or	Use amount, expiry, reference, customer, and reminder controls [3] [21] [27]

Razorpay surface	Revenue event	Proposed recovery value	Direct control boundary
		create a fresh approved link	
RazorpayX and business finance	Vendor, payroll, payout, and cash operations	Context for merchant cash stress and escalation	Treat this as context unless the project has explicit scopes for money movement [38] [38]

The design implication is to make the merchant the policy owner and Razorpay the payment-state system of record. The agent should recommend and execute only approved recovery actions, while the merchant sees why the case was selected, what was attempted, and whether the money was actually captured.

Why the Track Is a Direct Fit, with a Critical Scope Distinction

Key takeaway: The track maps almost exactly to Razorpay's transaction graph, but a strong entry must add orchestration and measurement rather than clone one existing agent.

The official track definition is unusually concrete: find revenue slipping away, detect revenue at risk, determine the right intervention, and execute a bounded workflow from payment failures and checkout abandonment to overdue receivables [8]. Its example directions include payment degradation and root-cause recovery, checkout drop-off, failed-subscription recovery, a B2B receivables chaser, mandate retry sequencing, Hinglish voice recovery, and promise-to-pay tracking [8]. The bar is not a dashboard that says revenue is at risk. It is measured money recovered, compliant escalation, stopping rules, and an audit trail [8].

Track requirement	What Razorpay evidence supports	Project interpretation
Detect risk	Payment failure is recorded as Failed, and Razorpay exposes payment and subscription webhook signals [14] [43] [24]	Build an event classifier and an aging/risk queue
Choose intervention	Checkout can be shown again after failure, subscriptions retry, and Payment Links support reminders [19] [45] [3]	Select among retry, customer-action request, reminder, link regeneration, or human review
Execute bounded workflow	Razorpay webhooks deliver JSON by HTTP POST, while the platform documents API keys in test and live modes [14] [36]	Use adapters with permission scopes, idempotency, approval gates, and stop conditions
Prove recovery	The brief explicitly asks for measured money recovered and an audit trail [8]	Maintain a recovery ledger and compare treatment outcomes with a holdout or replay baseline

There is a second strategic clue: Razorpay's public Agent Studio page already describes abandoned-cart conversion, a dispute responder, and subscription recovery [23] [23] [23]. Rebuilding a single cart reminder or a single subscription retry agent may therefore look like a narrower copy of an existing product direction. The differentiation should be a cross-surface recovery orchestrator that understands a common case model, chooses among interventions, attributes recovered value, and explains why it stopped.

This does not make the track redundant. It raises the bar. The buildathon can demonstrate the reasoning and reliability of the control plane even if the production Agent Studio has separate specialized agents. Your project can show how one merchant policy covers payment failures, checkout drop-offs, subscription

obligations, and receivables without allowing a language model to act outside its scope.

The right product thesis is: "Given a verified revenue-risk event, recover the most value with the least customer friction, while proving what happened and stopping safely." That thesis is narrower, more measurable, and more useful to Razorpay than "an AI that sends payment reminders."

Revenue-Leakage Baseline: Four Core Lanes, One Adjacent Lane

Key takeaway: Build a single normalized case model, but keep each recovery lane's trigger, action, and accounting rules separate.

The public documentation supports four core lanes. For ordinary payments, Razorpay records a bank-reported failed status as Failed [14], and its integration guidance says to notify the customer and display checkout again so the customer can retry [19]. For subscriptions, Razorpay says an automatic charge is retried and that a subscription moves to pending when retries are not exhausted; repeated failed charges can trigger another webhook [45] [24]. For checkout abandonment, Razorpay documents an abandoned-cart webhook for tracking drop-offs and retargeting customers [5]. For receivables, Razorpay supports invoices for sending invoices and accepting payments, while Payment Links expose amount, expiry, reference, description, customer details, and reminder management [27] [21] [3].

Lane	Detection signal	First intervention	Recovery proof	Important unknown or stop condition
Payment failure	<code>payment.failed</code> , failure status, order/payment state	Retry or reopen checkout only when the failure is eligible;	A subsequent captured payment tied to the	Stop if captured, already retried beyond policy, state

Lane	Detection signal	First intervention	Recovery proof	Important unknown or stop condition
		otherwise request a different permitted action	same order or case	is ambiguous, or the decline is not recoverable
Abandoned checkout	Abandoned-cart webhook or an order/session with no successful payment	Personalized, consented reminder or return-to-checkout link	A later paid order attributable to the case	Confirm the webhook payload and identity fields in the sandbox before designing the model [5]
Failed subscription	Failed auto-charge, pending state, repeated failure webhook	Retry sequence, payment-method update request, or human escalation	Successful subscription charge within a defined attribution window	Do not treat every pending state as permission for unlimited retries [45]
Overdue receivable	Unpaid invoice or payment link past a merchant-defined due threshold	Reminder, payment-link resend, promise-to-pay capture,	Payment applied to the original invoice or reference	Exact invoice status events, due-date semantics,

Lane	Detection signal	First intervention	Recovery proof	Important unknown or stop condition
		or human queue		and action APIs need verification [25] [46]
Dispute or refund, adjacent lane	Dispute webhook or refund event	Evidence preparation, merchant review, or reconciliation	Dispute outcome or payment not reversed	Do not count a refunded or disputed amount as ordinary recovery [28] [31]

A common case schema should include `case_id`, merchant, entity type, entity ID, amount and currency, created and due times, current state, failure class, attempt count, customer consent state, proposed action, approval state, action result, recovery attribution, and reversal status. The schema should preserve the original event and the current verified state so a later correction is possible.

The most important accounting choice is to separate opportunity from outcome. "At risk" is an estimate; "attempted" is an action; "recovered" is a verified capture or payment; and "incremental recovered" is the amount above what would have happened without the agent. A refund, chargeback, duplicate capture, or unrelated later payment must not inflate the result.

Buildable Architecture: State, Evidence, Action, Outcome

Key takeaway: Make the system event-driven and reconciliation-first; use AI after the payment state is trustworthy, not before.

The proposed architecture has six layers. First, a public webhook gateway receives Razorpay's HTTP POST JSON notifications [14]. It verifies signatures, validates test fixtures, and stores the raw event envelope. Razorpay documents signature verification and separately directs builders to read about idempotency and webhook event order [17] [41]. Second, an event normalizer maps payment, order, subscription, payment-link, invoice, refund, and dispute events into a common case model without losing source fields.

Third, a reconciliation worker reads the current entity state through the REST and JSON API surface, because webhook payloads are snapshots. Razorpay explicitly notes that a payment may move from authorized to captured quickly and that the authorized webhook payload represents the earlier state; captured payment also marks the order paid and appears through captured and order-paid events [43]. This is the central reliability problem. The agent should be able to say "the event was failed, but the current order is paid, so no recovery action is allowed."

Fourth, a risk and eligibility engine computes features such as amount, age, attempt count, payment or entity state, subscription status, link expiry, prior customer outcomes, and merchant policy. It should output a reason code and confidence, not an opaque instruction. Fifth, a bounded action layer exposes only approved tools: retry an eligible charge, reopen or send an approved payment path, manage a reminder, draft a message, or create a human task. Sixth, an immutable recovery ledger records every observation, decision, approval, tool call, result, recovery amount, reversal, and stop reason.

Layer	Minimum implementation	Failure it prevents
Ingestion	Signed webhook receiver, raw-event store, replay queue	Forged events and untestable processing [17]
State	Idempotent event key, ordering metadata, current-state read, reconciliation	Duplicate actions and stale-state recovery [41] [43]

Layer	Minimum implementation	Failure it prevents
Decision	Explainable score, reason code, policy version, confidence	Unreviewable or inconsistent choices
Action	Tool allowlist, dry-run mode, retry budget, approval gate	Unbounded retries and unauthorized money movement
Outcome	Capture lookup, invoice/reference linkage, refund/dispute exclusion	False claims of recovered revenue [31]
Evidence	Case timeline, message, tool arguments, human decision, audit export	Failure to meet the buildathon audit requirement [8]

Use a deterministic state machine for payment truth and an LLM only for bounded tasks such as summarizing a failure, selecting among approved playbooks, drafting a customer message, extracting a promise-to-pay date, or explaining a human queue. If the model cannot identify the entity, current state, allowed action, or consent status, it should abstain and escalate.

Intervention Design: Deterministic State Machine plus Bounded AI

Key takeaway: Optimize for expected net recovery, not the number of retries or messages sent.

A useful policy can score each eligible case as $\text{expected_net_recovery} = \text{probability_of_success} \times \text{amount} - \text{expected_cost} - \text{expected_customer_risk}$. The probability model can start with rules and later learn from outcomes. Stripe's public Smart Retries documentation provides a useful industry precedent: automatic retries can run on a configured schedule, while its AI chooses retry timing using dynamic signals and supports policies such as a specified

number of attempts within a defined period [18] [18]. That is a design reference, not evidence that Razorpay exposes the same model or all of the same signals.

For a payment failure, the agent should first confirm that the payment is still unpaid and that no later capture exists. It can then classify the failure using available status and error information, choose a retry or customer-action playbook, and stop after the merchant's attempt and time budget. For a subscription, the agent should respect Razorpay's existing retry and pending behavior, avoid competing with it, and trigger a customer-action or human workflow only when the policy says the next intervention is useful [45]. For an abandoned checkout, it should send a consented, low-pressure return path and learn whether the drop-off is associated with a technical failure, price friction, or an unresponsive customer. For an overdue payment link or invoice, it should prioritize by amount, age, prior promise, and customer relationship, then use a reminder or human queue rather than repeated generic messages.

Decision output	AI may do	AI may not do without explicit policy and approval
Eligibility	Summarize evidence and assign a reason code	Declare a payment recovered from a webhook alone
Prioritization	Rank cases by expected value and urgency	Change merchant thresholds silently
Retry	Recommend timing within a fixed budget	Retry hard or ambiguous cases indefinitely
Communication	Draft a clear message in an approved language and channel	Use false urgency, shame, threats, or an unapproved offer
Receivables	Extract a reply, promise date, or dispute reason	Admit liability, waive debt, or alter invoice terms autonomously
Escalation	Build a concise evidence packet	Bypass a merchant approval gate

HighRadius describes behavior-based collections as an alternative to one-size-fits-all email, using willingness-to-pay signals to choose timing, channel, and tone [22]. It reports a Ferrero Group deployment with a 28% decrease in DSO, a 67% reduction in average days delinquent, and more than 1,000 hours saved annually [22] [22]. These are vendor-reported case-study results, not a Razorpay benchmark, but they reveal the mechanism worth demonstrating: prioritize human attention and personalize the next action instead of increasing message volume.

The best agent therefore has an abstention state. A low-confidence case, conflicting webhook, high-value account, active dispute, opt-out, or missing consent becomes a human task with evidence. An intelligent no-op is a successful outcome when the alternative is a duplicate charge or a customer complaint.

Safety, Compliance, and Merchant Control

Key takeaway: In financial workflows, safety is part of product quality and should be visible in the demo.

Razorpay documents PCI-DSS certification for a payment service provider handling card data [15]. The project should therefore avoid raw card numbers, CVV, authentication secrets, and unnecessary personal data entirely. Use tokenized payment references, masked identifiers, synthetic fixtures, and scoped test credentials. The project should also treat an external AI platform as a separate data processor: Razorpay's privacy policy says AI integrations are optional, process information necessary for the requested action, and do not share user or merchant information with an AI platform except data requested by users or merchants [29] [29] [29]. It also says logs are retained for 12 months unless law states otherwise and that data submitted to an AI platform is governed by that platform's policy [29] [29].

Razorpay's Agent Studio principles provide a strong control model. Merchants approve the data an agent can access, the actions it can take, and where human approval is required; review-first mode lets the agent prepare work without sending or submitting it [47]. Every action passes through a validation layer, out-of-scope actions are blocked, all actions are logged, and irreversible actions require explicit merchant approval [47]. The principles also require consent before outbound calls, WhatsApp messages, or emails, and call for screening against dark patterns such as

false urgency, confirm shaming, manufactured scarcity, and subscription traps [47] [47].

Risk	Default control for the project	Evidence to show in the pitch
Duplicate or stale payment event	Idempotency key plus current-state reconciliation	A captured order causes a logged no-op
Excessive retry	Per-case attempt count, time window, and merchant-configured budget	The agent stops and explains why
Unauthorized action	Tool allowlist, scope check, review-first mode, explicit approval	A blocked tool call is visible in the audit trail
Harmful communication	Consent check, approved templates, dark-pattern classifier, opt-out	The system refuses manipulative wording [47] [47]
Privacy or credential leakage	Synthetic data, token references, secret manager, minimal context	The model prompt contains no card data
Refund, dispute, or chargeback confusion	Separate case types and accounting exclusions	A refund is removed from recovered-value reporting [31]

The MCP Server is relevant because Razorpay describes it as a protocol integration between payment APIs and AI tools [42]. However, the available documentation does not establish the exact actions, permissions, authentication modes, or safety boundaries. Treat MCP as an optional adapter and ask Razorpay for its approved tool schema rather than assuming that every API operation should be exposed to an agent.

Evaluation, Experiments, and the Five-Minute Demo

Key takeaway: The project should win on attributable money, reliability, and restraint, not on an impressive chat transcript.

The buildathon asks for measured money recovered, compliant escalation, stopping rules, and an audit trail, and asks applicants to show a public repository, architecture, and a five-minute pitch [8] [8]. Build a replayable simulator with anonymized or synthetic cases and a known ground-truth outcome. Each fixture should include the initial event, later state changes, customer response, eventual capture or non-payment, refunds, and the correct policy result. This makes the demo repeatable without requesting live financial access.

Metric	Proposed definition	Why it matters
At-risk value	Sum of eligible unpaid amounts at decision time	Measures opportunity, not success
Eligible rate	Eligible cases divided by all detected cases	Shows policy discipline
Recovery rate	Verified recovered cases divided by eligible cases	Shows operational effectiveness
Gross recovered value	Verified captured or paid amount linked to an action	Shows direct outcome
Incremental recovered value	Treatment outcome minus matched holdout or baseline outcome	Avoids credit for payments that would have happened anyway
Net recovered value	Incremental value minus refunds, discounts, fees, and communication cost	Aligns optimization with economics

Metric	Proposed definition	Why it matters
False-action rate	Actions later found unnecessary, duplicated, or inconsistent with state	Measures financial risk
Customer harm rate	Complaints, opt-outs, failed consent checks, or policy violations per contact	Measures trust cost
Automation and escalation	Cases completed safely versus sent to human review	Shows where AI judgment is useful
Audit completeness	Cases with source event, decision reason, tool result, outcome, and stop reason	Proves traceability

The demo sequence should be: ingest a failed payment; show classification and a bounded next action; inject a duplicate or out-of-order event; prove reconciliation prevents a second charge; replay an abandoned cart; show a consented message; process a pending subscription; route a high-value overdue receivable to review; then apply a refund or dispute event and show that the ledger corrects the recovery total. Finish with a dashboard showing gross, incremental, and net recovery plus no-op and escalation counts.

Use a small treatment-versus-holdout experiment in the simulator. The holdout need not send messages; it establishes how many cases recover naturally. If real traffic is unavailable, label all numbers as simulated and show the exact fixtures and calculation. Do not present a model's success probability as recovered revenue.

What the Project Should Get from Razorpay Before Building

Key takeaway: The biggest project risk is not model quality; it is lack of a trustworthy action and outcome contract.

Ask for the following in priority order. First, obtain a test-mode API key and the approved API and webhook scope. Razorpay documents API keys in test and live modes and REST/JSON responses [36], but the project should use test mode only unless Razorpay explicitly authorizes more. Second, request a webhook catalog and replay fixtures for payments, orders, subscriptions, abandoned carts, payment links, invoices, refunds, and disputes. The public pages establish that these surfaces exist, but several exact payload fields and invoice event meanings remain unspecified in the retrieved documentation [5] [46].

Third, request a safe action contract: which endpoint can retry a subscription charge, which endpoint can reopen or create a payment path, which reminder operation is allowed, what constitutes an idempotency key, and which actions require merchant approval. Fourth, request an anonymized dataset or simulator containing entity IDs, amounts, timestamps, states, failures, customer consent, eventual outcomes, refunds, and disputes. Without eventual outcomes, the project can rank risk but cannot honestly claim recovery.

Priority	Razorpay should provide	Why the builder needs it
P0	Test credentials, webhook secret, event fixtures, API docs, sandbox reset or replay ability	Build and verify the event loop without touching production money
P0	Current-state reads and approved action endpoints	Reconcile stale events and execute only real, permitted actions
P0	Ground-truth outcomes or a labeled simulator	Calculate recovered and incremental value
P1	Merchant policy schema: retry budget, amount threshold, due-age threshold, channels, consent, approval rules	Make the agent configurable rather than hard-coded
P1	Synthetic customer and merchant profiles plus failure taxonomy	Test personalization without exposing PII

Priority	Razorpay should provide	Why the builder needs it
P1	Communication sandbox for email, WhatsApp, or SMS	Demonstrate consent, templates, opt-out, and delivery outcomes
P2	Mentor feedback on target merchant segment and production constraints	Focus the scope and avoid duplicating an existing Agent Studio feature
P2	Approved MCP tool schema, if MCP is used	Prevent accidental exposure of broad payment operations [42]

If Razorpay cannot provide live APIs or outcome data, build a high-fidelity adapter interface with recorded fixtures and clearly label the result as simulation. That is better than fabricating production impact. The final deliverables should include the public repository, architecture diagram, event schema, policy file, replay command, metric calculation, failure cases, and a five-minute explanation of what the agent refused to do [8].

Synthesis

Key takeaway: The winning strategy combines Razorpay's stateful payment infrastructure, Stripe-style decision optimization, enterprise AR prioritization, and Razorpay's own merchant-control philosophy without importing any platform's assumptions uncritically.

Entity or strategy	Mechanism	Scope and time horizon	Trade-off and evidence base
Razorpay platform	Payment acceptance, recurring collections,	India-focused merchant transaction lifecycle, from	Strong first-party surface evidence, but exact payload and action

Entity or strategy	Mechanism	Scope and time horizon	Trade-off and evidence base
	invoices, payment links, webhooks, and business-finance APIs [7] [38]	immediate failure to receivable aging	permissions must be obtained
Razorpay Agent Studio	Specialized agents for abandoned-cart conversion, dispute response, and subscription recovery [23] [23] [23]	Merchant operations with ongoing monitoring and approval controls	Strong strategic signal and guardrail model; creates overlap that demands orchestration-level differentiation
Stripe Smart Retries	Automatic retries with configurable policy and ML-based timing signals [18] [18]	Subscription and invoice recovery over a retry window	Useful mechanism precedent, but platform-specific signals and hard-decline rules cannot be copied blindly
HighRadius and Ferrero case study	Behavior-based collections, channel and tone selection, worklist prioritization, and touchless dunning [22] [22]	Enterprise receivables over days or months	Vendor-reported outcome evidence; useful for prioritization, not a Razorpay benchmark

Entity or strategy	Mechanism	Scope and time horizon	Trade-off and evidence base
Proposed project	One case model, reconciled state, bounded playbooks, human review, and recovery ledger	Cross-surface recovery with immediate actions and longer receivable follow-up	More integration work, but it directly answers the track's measured-recovery and audit requirements [8]

The first tension is automation versus control. Smart retry logic rewards timely action, while Razorpay's guardrails require merchant permissions, approval for irreversible actions, and the ability to turn agents off [18] [47]. The resolution is not less automation; it is automation inside a visible policy envelope with attempt budgets and abstention.

The second tension is breadth versus evidence. The track names payment failure, checkout abandonment, subscription recovery, and overdue receivables, but the public documentation is richer for payment and subscription behavior than for exact abandoned-cart and invoice payloads [5] [45] [46]. Therefore, build the common model and simulator first, then activate each adapter only when its event and action contract is verified.

The third tension is gross recovery versus incremental recovery. A later payment is not automatically proof that the agent caused recovery, and a successful capture can later be refunded. A trustworthy project reports the full chain: eligible opportunity, action, verified capture, attribution window, reversal status, cost, and net incremental value.

The final recommendation is to build a "Recovery Control Plane" with payment-failure recovery as the reliable core, abandoned-checkout recovery as the visible growth case, subscriptions and payment links as extensions, and disputes/refunds as a reconciliation lane rather than an ordinary retry lane. Let deterministic code own payment truth, let AI own bounded reasoning and communication drafts, and let merchants own permissions and irreversible decisions. That design is useful to Razorpay even if the production product eventually splits the work into several specialized agents.

References

1. What is PCI DSS Compliance? 12 Basic Requirements and
<https://razorpay.com/blog/what-is-pci-dss-compliance/>
2. Razorpay Becomes One of the First Payment Gateways to
<https://razorpay.com/newsroom/razorpay-becomes-indias-first-payment-gateway-to-launch-mcp-server-for-instant-ai-payment-integration/>
3. Manage Reminders for Payment Links.
<https://razorpay.com/docs/api/payments/payment-links/reminders/?preferred-country=US>
4. Payment Retries.
<https://razorpay.com/docs/payments/subscriptions/payment-retries/>
5. Abandoned Cart Webhook | Razorpay Magic Checkout.
<https://razorpay.com/docs/payments/magic-checkout/abandoned-cart/?preferred-country=US>
6. Razorpay - Best Payment Solution for Online Payments India.
<https://razorpay.com/>
7. <http://razorpay.com/about>*. <http://razorpay.com/about>
8. Razorpay AI Buildathon — Build. Show. Get hired..
<https://razorpay.com/buildathon/>
9. Create a Standard Payment Link - Razorpay Docs.
<https://razorpay.com/docs/api/payments/payment-links/create-standard/?preferred-country=US>
10. Failed Payments and Involuntary Churn — A Definitive Guide.
<https://www.chargebee.com/resources/guides/involuntary-churn-payment-failed/>
11. Payments Webhook Events.
<https://razorpay.com/docs/webhooks/payments/?preferred-country=US>
12. Razorpay Docs. <http://razorpay.com/docs>
13. Automating Accounts Receivable with AI agents in 2026.
<http://paraglide.ai/blog/how-ai-agents-automate-accounts-receivable>
14. About Webhooks. <https://razorpay.com/docs/webhooks/>

15. About Razorpay Security. <https://razorpay.com/docs/security/?preferred-country=US>
16. List of Errors. <https://razorpay.com/docs/errors/payments/list/?preferred-country=US>
17. Webhooks | Best Practices. <https://razorpay.com/docs/webhooks/best-practices/?preferred-country=US>
18. Automate payment retries. <http://docs.stripe.com/billing/revenue-recovery/smart-retries>
19. Integration Steps. <https://razorpay.com/docs/payments/payment-gateway/web-integration/standard/integration-steps/>
20. Payment Retries by Chargebee — Revive + Smart Dunning. <https://www.chargebee.com/payments/retries-and-dunning/>
21. Payment Links APIs. <https://razorpay.com/docs/payments/payment-links/apis/?preferred-country=US>
22. 7 Real-World Use Cases Of AI In Accounts Receivable 2026. <http://highradius.com/resources/Blog/ai-in-accounts-receivable>
23. Razorpay Agent Studio | AI Agents for Payments & Revenue Operations. <http://razorpay.com/agent-studio>
24. Subscriptions Webhook Events. <https://razorpay.com/docs/webhooks/subscriptions/?preferred-country=US>
25. Update an invoice - Razorpay Docs. <https://razorpay.com/docs/api/payments/invoices/update/?preferred-country=US>
26. Revenue recovery. <https://docs.stripe.com/billing/revenue-recovery>
27. Invoice APIs. <https://razorpay.com/docs/api/payments/invoices/?preferred-country=US>
28. Disputes | Subscribe to Webhooks. <https://razorpay.com/docs/payments/disputes/subscribe-to-webhooks/?preferred-country=US>
29. <http://razorpay.com/privacy-policy>*. <http://razorpay.com/privacy-policy>
30. About Disputes | Razorpay Docs. <https://razorpay.com/docs/payments/disputes/?preferred-country=US>

31. Refunds Webhook Events. <https://razorpay.com/docs/webhooks/refunds/?preferred-country=US>
32. How we built it: Smart Retries. <https://stripe.com/blog/how-we-built-it-smart-retries>
33. How to Reduce RTO in Ecommerce India Using AI (2026
<https://www.buildfastwithai.com/blogs/razorpay-agent-studio-ai-payment-platform>
34. Subscriptions | Notifications.
<https://razorpay.com/docs/payments/subscriptions/notifications/?preferred-country=US>
35. Agent Studio: AI Agents by Razorpay. <https://razorpay.com/blog/agent-studio-ai-agents-by-razorpay/>
36. Razorpay API Documentation | Razorpay Docs.
<http://razorpay.com/docs/api?preferred-country=US>
37. About Errors | Razorpay Docs. <https://razorpay.com/docs/errors/?preferred-country=US>
38. RazorpayX - The All-In-One Business Banking Suite. <http://razorpay.com/x>
39. Subscribe to Webhooks.
<https://razorpay.com/docs/sg/payments/subscriptions/subscribe-to-webhooks>
40. Payment Links Webhook Events.
<https://razorpay.com/docs/webhooks/payment-links/?preferred-country=US>
41. Validate and Test Webhooks. <https://razorpay.com/docs/webhooks/validate-test/?preferred-country=US>
42. About Razorpay MCP Server. <https://razorpay.com/docs/mcp-server/?preferred-country=US>
43. Payments Webhook Events | Razorpay Docs.
<http://razorpay.com/docs/webhooks/payments>
44. Razorpay Agent Studio — Terms and Conditions.
<https://razorpay.com/tnc/agent-studio/>
45. Notifications.
<https://razorpay.com/docs/us/payments/subscriptions/notifications>

46. Invoices Webhook Events. <https://razorpay.com/docs/webhooks/invoices/?preferred-country=US>
47. Razorpay Agent Studio: Principles, Guardrails, and Merchant Control.
<http://razorpay.com/blog/razorpay-agent-studio-principles-guardrails-and-merchant-control>
-